DK 26

Технологии

Налог на безопасность:

чем мы платим за защиту Kubernetes и как сделать «налоговый вычет»

Никита Ладошкин

Руководитель разработки PT Container Security в Positive Technologies

Никита Ладошкин

PT Container Security

12 лет в ИТ

ИБ, облака, container security

Переводил продовые сервисы
на Kubernetes еще с 1.8

Цель доклада

Показать экономику runtime-защиты

«Ванильный» Kubernetes закрывает оркестрацию, но не безопасность

Что обычно остается незащищенным

Примеры

- Контроль runtime-поведения процессов
- Работа с секретами и сервис-аккаунт токенами
- Lateral movement между namespace/нодами
- Аномальная сетевая активность

Откуда берется «налог на ИБ»

Чтобы закрыть runtime-риск, приходится достраивать технологии, процессы и роли.

Из чего складывается цена на управляемую безопасность?

Сценарий «где-то я это видел»

Небольшая команда

Критичные сервисы в Kubernetes

Защитить без заморозки релизов

Что уже есть

Оптимистично

› Образы сканируются в CI

› IaC
Чек на мисконфиги

› RBAC
Ручные ревью

Налог №1: компетенции

Налог №1

Компетенции

Runtime-защита
не включается
одной кнопкой

Платим временем специалистов

- Выбор стека
- Настройка детекторов
- Интеграция с SOC

Платим вниманием специалистов

- Фолсы
- Исключения
- Обучение команд

Кому принадлежит процесс

SOC
Мониторинг

DevOps
Внедрение

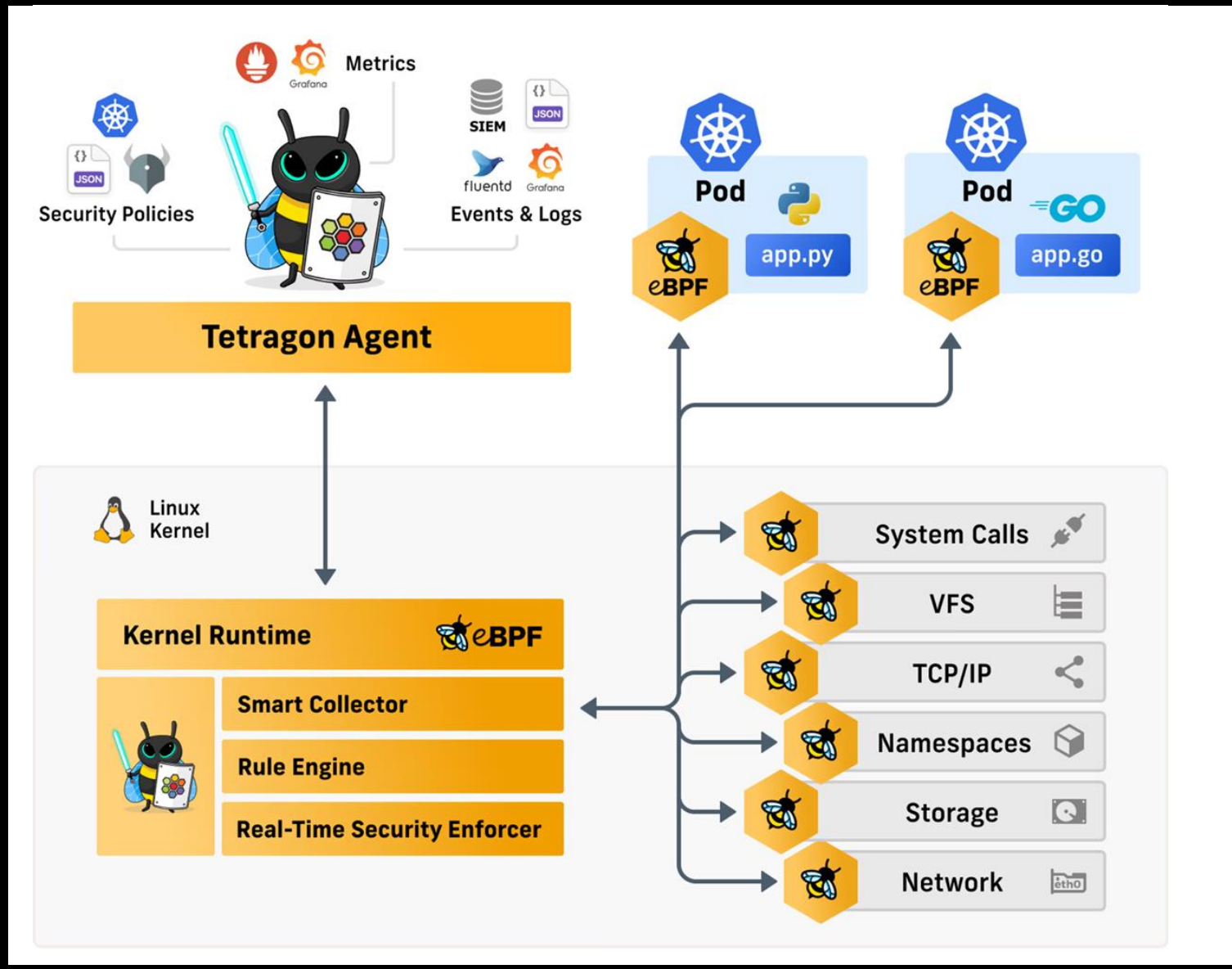
Platform
Масштабирование

Разработка
Контекст

Security
Правила

Менеджмент
SLA и приоритеты

Сложность технологии добавляет работы



Разберем статьи расходов и где можно срезать чек без потери защиты

Как выбирать инструмент, чтобы не переплатить

Точность детекта

Сколько полезных инцидентов на 1000 событий

tetragon

Совместимость

SIEM, лог-шина, storage и процессы SOC

RUNTIME RADAR

Стоимость владения

Лицензия, ресурсы и часы команды на сопровождение

falco

Почему open-source экономит

Готовые правила

Быстрый старт

ТЫК ↗

Комьюнити

Быстрые апдейты

Прозрачная логика

Проще аудит

Rules as Code

Git + review

Пилот без долгой закупки

GitOps rollout

Меньше рутины

runtime-radar / event-processor / detector / wasm / CS_RT_DOWNLOAD_TOOLS / **main.go**

Code | Blame | 425 lines (395 loc) · 13 KB

```go
90  func (d Detector) TriggerCriteria(ctx context.Context, req *api.TriggerCriteriaReq) (*api.TriggerCriteriaResp, error)
101
102 func (d Detector) Detect(ctx context.Context, req *api.DetectReq) (*api.DetectResp, error) {
103     // Detector info added to DetectResp because detector info is always correlated to response, thus
104     // to avoid +1 Wasm call on detect.
105     resp := &api.DetectResp{
106         // Default response indicates that nothing detected (this is redundant and put here just for reference,
107         // as Severity == api.DetectResp_NONE == 0 when omitted (default zero value)).
108         Severity: api.DetectResp_NONE,
109     }
110
111     event := req.GetEvent().GetEvent()
112
113     switch ev := event.(type) {
114     case *tetragon.GetEventsResponse_ProcessExec:
115         // Nothing here
116     case *tetragon.GetEventsResponse_ProcessExit:
117         // Nothing here
118     case *tetragon.GetEventsResponse_ProcessKprobe:
119         kprobe := ev.ProcessKprobe
120
121         binary := kprobe.GetProcess().GetBinary()
122         args := kprobe.GetProcess().GetArguments()
123         function := kprobe.GetFunctionName()
124
125         if function != "tcp_connect" {
126             return resp, nil
127         }
128
```

Куда внедрять в периметре

Архитектура зависит от реальности: on-prem/облако, сеть, compliance и зрелость SOC

Prod first
Критичные сервисы

Stage next
Обработка правил

Dev selective
Минимум паранойи

Как харденинг режет расходы

- Запрет shell/exec
- Запрет privileged

Меньше площадь атаки

- Read-only FS

Разбор инцидентов короче

- Drop caps
- Distroless

Меньше лишних системных вызовов

Налог №2: вычислительные ресурсы

Налог №2: вычислительные ресурсы

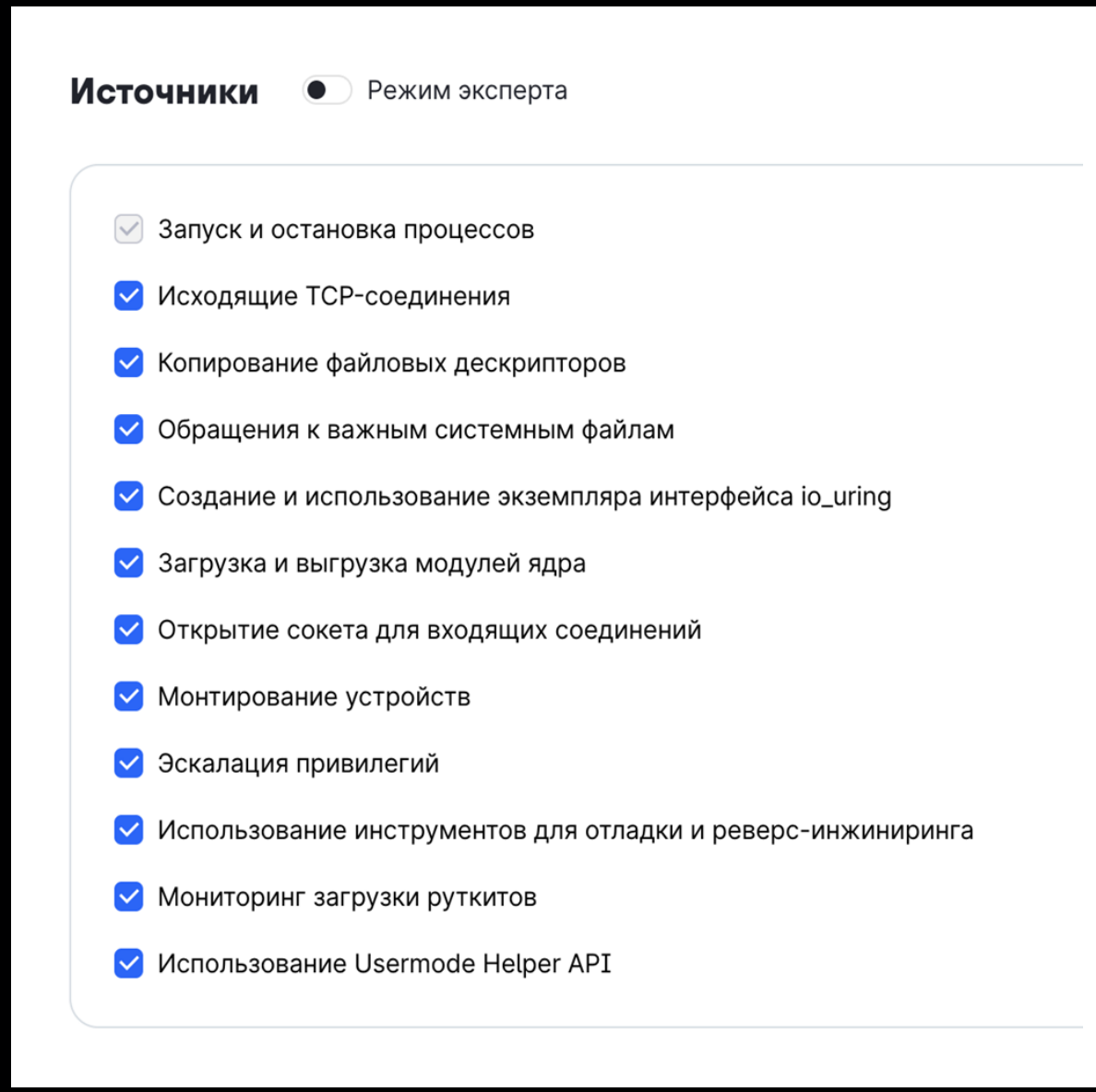
В runtime-мониторинге нельзя заложить “X CPU и Y RAM”

Нагрузка нелинейная и меняется от сценария к сценарию

Почему нагрузка непредсказуема

На объем событий одновременно влияют платформа, стек и текущий режим работы кластера

- Приложения на Python обычно шумнее Go
- Источники сигналов process/file/network дают разный поток
- Режим работы релизы, debug, инциденты = пики

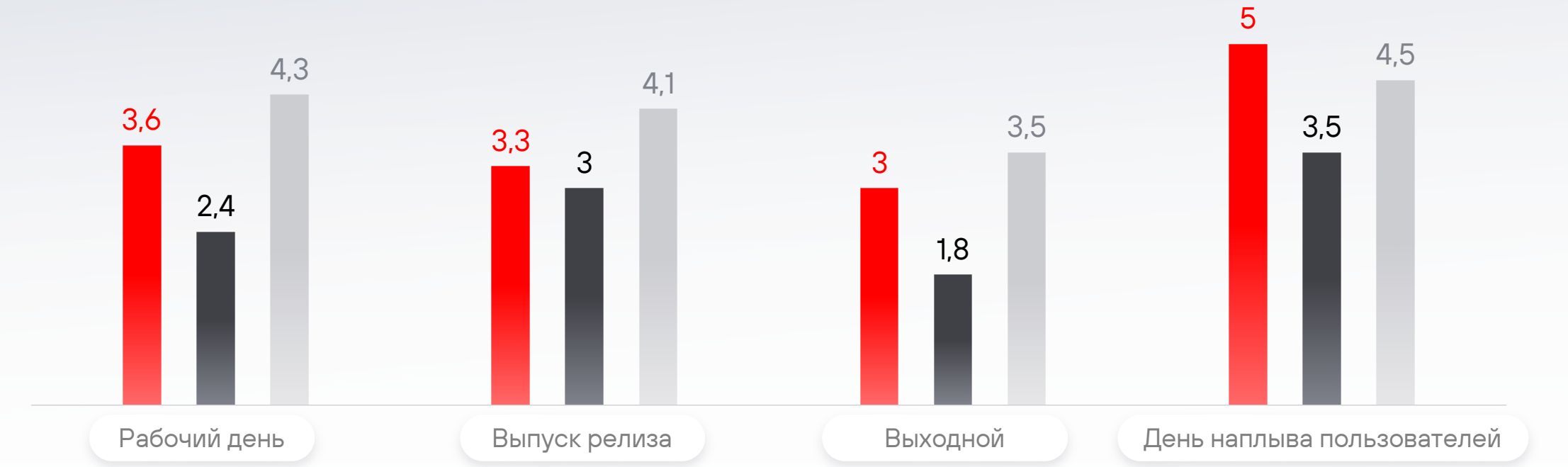


1 действие в контейнере! = 1 событие

💥	default/example-w...	—	/usr/bin/cat	/proc/mounts	⚠	⊘	4/3/2026, 18:05:01.286913907
🧪	default/example-w...	security_file_permi...	/usr/bin/cat	/proc/mounts	⚠	⊘	4/3/2026, 18:05:01.286788495
🧪	default/example-w...	security_file_permi...	/usr/bin/cat	/proc/mounts	⚠	⊘	4/3/2026, 18:05:01.286716778
🧪	default/example-w...	security_mmap_file	/usr/bin/cat	/proc/mounts	⚠	⊘	4/3/2026, 18:05:01.286368164
🚀	default/example-w...	—	/usr/bin/cat	/proc/mounts	⚠	⊘	4/3/2026, 18:05:01.286057120
💥	default/example-w...	—	/usr/bin/cat	/proc/mounts	⚠	⊘	4/3/2026, 18:04:55.171261310

Одна команда часто рождает цепочку
process + file + network

Runtime-мониторинг != фиксированная нагрузка



База
process only

Норма
process + file

Пик
+network + debug

Нагрузочные тесты

только цифры

10-100k

событий/сек.
на нескольких нодах

20-60

событий
на один exec

-40%

от всех событий
после ранней
фильтрации

Что можно не мониторить в первую очередь

Dev/test сервисы

Конечные таски без внешнего траффика

Стабильные системные контейнеры

Временные sandbox-кластеры

Вредряйтесь продуманно

Risk-based подход:
не все одинаково критично

Сначала защищаем prod, data plane и внешние API

Фильтруйте как можно раньше:
на eBPF-сенсоре, а не только в SIEM

Планируйте сценарии, а не «средний» кластер

Расширяйте источники только после замера эффекта

Каждый сценарий =
свой набор сигналов =
свой расход CPU/RAM/IO

Process only - базовый расход

Process+file+network - кратный рост

Масштабируйтесь по сигналу

1

Соберите baseline:
скорость событий, queue lag

2

Алерты на drops
и latency

3

Autoscaling
по security-метрикам

4

Retention по SLA

5

Пересмотр порогов
каждый релиз

Налог №3: время

Налог №3: время

Внедрение сенсоров

Настройка правил

Тестирование реакций

Разбор фолсов

Март | Апрель | Май | Июнь | Июль | Август | Сентябрь

Исключения по командам

Синхронизация Dev/Sec/Ops

Обновления без простоя

Выводы в backlog

Шум

Триаж

Ложные срабатывания

Проверка контекста

Debug в prod

Где теряется время при реагировании

Постмортем

Контроль регрессии

Обновление правил

Фиксация исключений

Решение по инциденту

Коммуникация с разработкой

Как срезать косты операционкой

Рабочий цикл

Ежедневно
triage high/critical и контроль потока

Еженедельно
ревью шумных правил

Ежемесячно
правка исключений и ретест блокировок на stage

Практический эффект по проектам внедрения

↓ 25–35%
ложных эскалаций за 1-2 месяца

<2 ч
MTTR по critical

Налог №4: ошибки

Налог №4: ошибки

Цена проактивной ошибки

Ложная блокировка
может остановить сервис

Цена пассивной ошибки

Слепая зона
пропускает атаку

Enforcement без боли для продакшена

Требование к стеку

Встроиться в текущий SIEM/очередь без отдельного «зоопарка» инструментов

Шаг 1

Наблюдаем
1-2 недели

Шаг 2

Alert + ручная валидация

Шаг 3

Block только
на подтвержденных паттернах

Шаг 4

Rollback-кнопка и owner
на каждое правило

Как срезать косты

метрики и изоляция стека

Нагрузка сенсоров

< 1500m
P95 CPU

< 1500Mi
memory

< 1%
drop-rate

Конвейер событий

< 1 мин.
queue lag

0 на critical
потери

30/90/365
retention

Изоляция инфраструктуры

CH/RabbitMQ/PG
вне защищаемого prod-кластера

Выводы

Kubernetes «из коробки» не дает runtime-защиту

Экономия начинается с приоритизации и харденинга

Налог неизбежен: люди, ресурсы, время, ошибки

Регулярный процесс дешевле, чем героизм на инциденте

Технологии

Спасибо за внимание!

DK 26 Технологии

Оцените доклад

Никиты Ладошкина